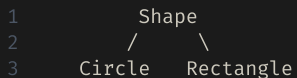


C++ 프로그래밍

상속 (Inheritance)

상속이란?

기존 클래스(기본 클래스)의 멤버를 물려받아 새 클래스(파생 클래스)를 만드는 기법이다.



- **코드 재사용**: 공통 멤버를 기본 클래스에 한 번만 작성
- **is-a 관계**: "Circle은 Shape이다" — 파생 클래스는 기본 클래스의 일종
- **계층 확장**: 공통 인터페이스를 유지하면서 기능을 추가

```
class Shape {                // 기본 클래스 (Base Class)
public:
    double x, y;
    Shape(double _x, double _y) : x(_x), y(_y) {}
    void print() const { printf("위치: (%.1f, %.1f)\n", x, y); }
};

class Circle : public Shape { // 파생 클래스 (Derived Class)
    double radius;
public:
    Circle(double _x, double _y, double r) : Shape(_x, _y), radius(r) {}
    double area() const { return 3.14159 * radius * radius; }
};
```

기본 문법

class 파생 : public 기본 형태로 상속함. 파생 클래스는 기본 클래스의 public / protected 멤버를 그대로 사용 가능.

클래스 정의

```
class Shape {
public:
    double x, y;
    Shape(double _x, double _y) : x(_x), y(_y) {}

    double area() const { return 0.0; }
    void print() const { printf("넓이: %.2f\n", area()); }
};

class Circle : public Shape {
    double r;
public:
    Circle(double _x, double _y, double _r)
        : Shape(_x, _y), r(_r) {} // 기본 클래스 생성자 호출

    double area() const { return 3.14159 * r * r; }
};

class Rect : public Shape {
    double w, h;
public:
    Rect(double _x, double _y, double _w, double _h)
        : Shape(_x, _y), w(_w), h(_h) {}

    double area() const { return w * h; }
};
```

사용

```
Circle c(1.0, 2.0, 5.0);
Rect r(0.0, 0.0, 3.0, 4.0);

// 상속된 멤버 사용
printf("x=%.1f\n", c.x); // Shape의 x
c.print(); // Shape의 print() → area() 호출

// 파생 클래스 멤버
printf("%.2f\n", c.area()); // Circle::area
printf("%.2f\n", r.area()); // Rect::area
```

print() 는 Shape 에 한 번만 정의했지만 Circle , Rect 모두에서 동작한다.

private 멤버

기본 클래스의 `private` 멤버는 파생 클래스에서도 직접 접근할 수 없다.

접근하려면 기본 클래스가 제공하는 `protected` 또는 `public` 인터페이스를 통해야 한다.

❌ 직접 접근 — 컴파일 오류

```
class Account {
    double balance;    // private
public:
    Account(double _balance) : balance(_balance) {}
};

class SavingsAccount : public Account {
public:
    SavingsAccount(double _balance)
        : Account(_balance) {}

    void addInterest(double rate) {
        balance *= (1 + rate);    // ❌ private 접근 불가
    }
};
```

✅ 인터페이스를 통한 접근

```
class Account {
    double balance;    // private
public:
    Account(double _balance) : balance(_balance) {}

    double getBalance() const { return balance; }
    void deposit(double amount) { balance += amount; }
};

class SavingsAccount : public Account {
public:
    SavingsAccount(double _balance)
        : Account(_balance) {}

    void addInterest(double rate) {
        // public 인터페이스를 통해 간접 접근
        deposit(getBalance() * rate);    // ✅
    }
};
```

protected 멤버

protected 는 파생 클래스 내부에서만 접근 가능하다.
private 은 파생 클래스에서도 접근 불가.

```
class Shape {
private:
    int id;          // 파생 클래스에서 접근 불가
protected:
    double x, y;     // 파생 클래스 내부에서 접근 가능
public:
    Shape(double _x, double _y) : id(0), x(_x), y(_y) {}
};

class Circle : public Shape {
    double r;
public:
    Circle(double _x, double _y, double _r)
        : Shape(_x, _y), r(_r) {}

    void move(double dx, double dy) {
        // id += 1; // ❌ private - 접근 불가
        x += dx;    // ✅ protected - 파생 클래스 OK
        y += dy;    // ✅
    }
};
```

접근 지정자	같은 클래스	파생 클래스	외부
public	✅	✅	✅
protected	✅	✅	❌
private	✅	❌	❌

```
Circle c(0, 0, 5);
// c.x = 1.0; // ❌ protected - 외부 접근 불가
c.move(1.0, 2.0); // ✅ public 함수로 간접 수정
```

설계 원칙: 파생 클래스가 직접 수정해야 하는 멤버는 protected , 그 외에는 private 으로 선언한다.

상속 접근 지정자

상속 시 지정자(`public` / `protected` / `private`)에 따라 기본 클래스 멤버의 접근 수준이 바뀐다.

```
class Base {  
    public:    int pub;  
    protected: int prot;  
    private:  int priv; // 항상 접근 불가  
};  
  
class PubDerived : public Base {};  
class ProtDerived : protected Base {};  
class PrivDerived : private Base {};
```

기본 클래스 멤버	<code>public</code> 상속	<code>protected</code> 상속	<code>private</code> 상속
<code>public</code>	<code>public</code>	<code>protected</code>	<code>private</code>
<code>protected</code>	<code>protected</code>	<code>protected</code>	<code>private</code>
<code>private</code>	접근 불가	접근 불가	접근 불가

```
PubDerived pd;  
pd.pub = 1; // ✅ public 상속 → pub은 public 유지  
  
ProtDerived td;  
// td.pub = 1; // ❌ protected 상속 → pub이 protected로 강등
```

일반적으로 `public` 상속만 사용한다. `protected` / `private` 상속은 "구현 상속"으로, 드물게 쓰인다.

생성자 & 소멸자 호출 순서

파생 클래스 객체를 만들 때 기본 클래스 생성자가 먼저 호출된다. 소멸은 반대 순서다.

```
class Base {
public:
    Base() { printf("Base 생성\n"); }
    ~Base() { printf("Base 소멸\n"); }
};

class Derived : public Base {
public:
    Derived() { printf("Derived 생성\n"); }
    ~Derived() { printf("Derived 소멸\n"); }
};

int main() {
    Derived d;
}
```

실행 결과

```
1   Base 생성
2   Derived 생성
3   Derived 소멸
4   Base 소멸
```

```
1   생성: Base → Derived (기반 먼저)
2   소멸: Derived → Base (역순)
```

생성자는 가장 깊은 기반 클래스부터, 소멸자는 가장 바깥 파생 클래스부터 호출된다.

기본 클래스 생성자 호출

파생 클래스 생성자의 초기화 목록에서 기본 클래스 생성자를 명시적으로 호출할 수 있다.

```
class Shape {
public:
    double x, y;
    Shape(double _x, double _y) : x(_x), y(_y) {
        printf("Shape(%g, %g) 생성\n", _x, _y);
    }
};

class Circle : public Shape {
    double r;
public:
    //          ↓ 기본 클래스 생성자 호출
    Circle(double _x, double _y, double _r)
        : Shape(_x, _y), r(_r) {    // Shape 먼저 초기화
        printf("Circle(r=%g) 생성\n", _r);
    }
};

class Cylinder : public Circle {
    double h;
public:
    Cylinder(double _x, double _y, double _r, double _h)
        : Circle(_x, _y, _r), h(_h) {    // Circle → Shape 순으로 초기화
        printf("Cylinder(h=%g) 생성\n", _h);
    }
};
```

초기화 목록에서 기본 클래스 생성자를 호출하지 않으면 **기본 생성자가 자동 호출된다**. 기본 생성자가 없으면 컴파일 오류.

함수 오버라이딩 vs 함수 숨김

파생 클래스에서 기본 클래스와 같은 이름의 함수를 정의하면 기본 클래스 버전이 **가려진다**(hiding).

함수 숨김 (virtual 없이)

```
class Shape {
public:
    void describe() const {
        printf("Shape\n");
    }
    double area() const { return 0.0; }
};

class Circle : public Shape {
    double r;
public:
    Circle(double _r) : Shape(0,0), r(_r) {}

    // Shape::area를 가린다 (hiding)
    double area() const {
        return 3.14159 * r * r;
    }
};
```

호출 결과

```
Circle c(5.0);

c.area();           // Circle::area   ✓
c.Shape::area();    // Shape::area - 명시적 호출

Shape* p = &c;
p->area();           // ⚠ Shape::area 호출!
                    // 포인터 타입 기준 - 컴파일 타임 결정
```

방식

결정 시점

방법

함수 숨김

컴파일 타임

포인터/참조 타입 기준

오버라이딩

런타임

`virtual` + 실제 객체 타입 기준

포인터/참조를 통한 다형성이 목적이라면 `virtual` 을 사용해야 한다.

업캐스팅 & 다운캐스팅

업캐스팅: 파생 → 기본 방향 변환. 항상 안전하며 암묵적으로 수행된다.

다운캐스팅: 기본 → 파생 방향 변환. 명시적 변환이 필요하고 주의가 필요하다.

업캐스팅 (Upcasting)

```
Circle c(0, 0, 5.0);

// 암묵적 업캐스팅 - 항상 안전
Shape* sp = &c;
Shape& sr = c;

sp->print(); // Shape::print 호출
// sp->area(); // ❌ Shape*로는 Circle 멤버 접근 불가
```

static_cast 다운캐스팅

```
Shape* p = new Circle(0, 0, 5.0);

// 타입이 맞다고 확신할 때만 사용
Circle* cp = static_cast<Circle*>(p);
printf("%.2f\n", cp->area()); // ✅

Shape* p2 = new Rect(0, 0, 3.0, 4.0);
Circle* wrong = static_cast<Circle*>(p2);
// ⚠️ 컴파일은 되지만 undefined behavior!
```

dynamic_cast 다운캐스팅

```
// dynamic_cast: 런타임에 타입 검사 (virtual 함수 필요)
Shape* p = new Circle(0, 0, 5.0);

Circle* cp = dynamic_cast<Circle*>(p);
if (cp) {
    printf("Circle: %.2f\n", cp->area()); // ✅
} else {
    printf("Circle이 아님\n");
}

Rect* rp = dynamic_cast<Rect*>(p);
// rp == nullptr - 타입 불일치
```

	static_cast	dynamic_cast
검사 시점	컴파일 타임	런타임
실패 시	UB	nullptr
속도	빠름	느림
요구 사항	—	virtual 함수

다중 상속 (Multiple Inheritance)

C++는 여러 기본 클래스를 동시에 상속할 수 있다.

문법

```
class Flyable {
public:
    void fly() { printf("날기\n"); }
};

class Swimmable {
public:
    void swim() { printf("수영\n"); }
};

// 두 클래스를 동시에 상속
class Duck : public Flyable,
             public Swimmable {
public:
    void quack() { printf("꽹꽹!\n"); }
};

int main() {
    Duck d;
    d.fly();    // Flyable::fly
    d.swim();   // Swimmable::swim
    d.quack();
}
```

다이아몬드 문제

```
class Animal { public: int age; };
class Lion : public Animal {};
class Tiger : public Animal {};

// Lion과 Tiger 각각 Animal을 상속
// → Duck에 Animal이 두 개 존재!
class Liger : public Lion, public Tiger {};

Liger li;
// li.age = 5;           // ✗ 모호함 - 어느 age?
li.Lion::age = 5;        // ✓ 명시적 지정
li.Tiger::age = 5;       // ✓
```

해결책: `virtual` 상속으로 기본 클래스를 하나만 공유

```
class Lion : virtual public Animal {};
class Tiger : virtual public Animal {};
class Liger : public Lion, public Tiger {};
// 이제 age가 하나만 존재
```

요약

개념	문법	핵심
상속	<code>class D : public B</code>	is-a 관계, 멤버 재사용
protected	<code>protected:</code>	파생 클래스 내부에서만 접근 가능
상속 접근 지정자	<code>public</code> / <code>protected</code> / <code>private</code>	멤버 접근 수준 변환; 보통 <code>public</code> 사용
생성자 순서	초기화 목록 : <code>Base(args)</code>	기본 → 파생 순 생성, 역순 소멸
함수 숨김	같은 이름 재정의	포인터 타입 기준 컴파일 타임 결정
업캐스팅	암묵적 변환	파생 → 기본, 항상 안전
다운캐스팅	<code>static_cast</code> / <code>dynamic_cast</code>	기본 → 파생, 주의 필요
다중 상속	<code>class D : public A, public B</code>	다이아몬드 문제 시 <code>virtual</code> 상속

상속은 is-a 관계일 때만 사용한다. 단순 코드 재사용이 목적이려면 포함(has-a)이 더 적합하다.